

Efficient 3D Gaussian Splatting with Axis-Shared Rasterization and Order-independent Transmittance

Zhican Wang*, Guanghui He*, Lingjun Gao[†], Dantong Liu[†], Shell Xu Hu[§],
Chen Zhang*, Zhuoran Song*, Nicholas Lane[†], and Hongxiang Fan[‡]

*Shanghai Jiao Tong University {wang_zhican, guanghui.he, chenzhang, songzhuoran}@sjtu.edu.cn

[†]University of Cambridge liudt921115@gmail.com, ndl32@cam.ac.uk

[‡]Imperial College London {lingjun.gao24, hongxiang.fan}@imperial.ac.uk

[§]Samsung AI shell.hu@samsung.com

Abstract—3D Gaussian Splatting (3DGS) has emerged as a powerful technique for novel view synthesis, combining high-quality reconstruction with efficient rendering. It has been widely adopted in domains such as AR/VR, robotics, and autonomous driving. However, achieving real-time performance on resource-constrained platforms remains challenging due to strict power and area budgets. Prior accelerators improve hardware performance but still overlook key inefficiencies, including insufficient rasterization efficiency, poor sorting scalability, and pipeline imbalance. This paper presents an architecture–algorithm co-design to address these challenges. First, we propose *axis-shared rasterization*, which precomputes and reuses common terms along the X- and Y-axes, reducing multiply-and-accumulate (MAC) operations by up to 38% while preserving high parallelism. Second, we develop a novel *order-independent transmittance* method that removes the need for explicit sorting by leveraging a lightweight Multilayer Perceptron (MLP) to directly approximate the transmittance of each Gaussian, enabling efficient α blending with negligible quality loss. Third, we design a *unified reconfigurable PE array* that supports both rasterization and MLP inference, sustaining high utilization without costly sorting hardware. Our experiments demonstrate that our design preserves rendering quality while achieving a $1.33 \sim 1.88\times$ speedup over the state-of-the-art 3DGS accelerators. Our code is open source at https://github.com/WangZhican/ISCA26_3DGS_Acc.

I. INTRODUCTION

3D Gaussian Splatting (3DGS) [22] has emerged as a prominent technique for novel view synthesis, offering both high-quality reconstruction and efficient rendering performance. It has been widely adopted across diverse domains, including robotics [56], augmented and virtual reality (AR/VR) [45], [55], and autonomous driving [23]. In contrast to neural radiance fields (NeRF) [32], which implicitly represent 3D scenes using neural networks, 3DGS explicitly encodes scenes as a large set of 3D Gaussians with learnable positions, sizes, shapes, colors, and opacities. Owing to its lower algorithmic complexity, 3DGS achieves significantly faster rendering performance than NeRF, making it particularly suitable for interactive applications. However, achieving real-time 3DGS rendering on resource-constrained platforms, such as edge GPUs, remains a significant challenge. For instance, on an

This work was supported by the National Natural Science Foundation of China under Grant U25B2057 and Grant 92464302. Corresponding author: Guanghui He.

TABLE I
COMPARISON WITH RELATED 3DGS ACCELERATORS.

	GSCore [25]	Meta. [29]	GBU [52]	Ours
Ras. MAC Reduction	✗	✗	✓	✓
Parallel Rasterization	✓	✓	✗	✓
Sorting Implementation	Hier. Bitonic	Hier. Bitonic	GPU-based	Replaced

NVIDIA Jetson Orin Nano edge GPU [38], we observe only approximately 20 frames per second on the MipNeRF-360 dataset [1]. The stringent power and area constraints of AR/VR edge devices further exacerbate the difficulty of deploying 3DGS in practice.

Given a camera pose as input, 3DGS renders a scene represented by 3D Gaussians (ellipsoids) into a final image, as illustrated in Fig. 1(a). The rendering pipeline comprises three essential steps: projection, sorting, and rasterization (details are provided in Sec. II-A). Profiling results, presented in Fig. 2 (left), indicate that rasterization and sorting dominate the overall latency, consistent with prior studies [11], [25], [29], [52]. Existing approaches, including GSCore and MetaSapiens [25], [29], enhance efficiency via Gaussian count reduction but neglect the inherent computational redundancy in rasterization. GBU [52] decreases rasterization MACs using spatial transformations and sequential differential computation; however, this introduces inter-pixel dependencies, thereby compromising pixel-level parallelism. Another work, Lumina [11], reduces the computation of sorting and rasterization by exploiting the similarity between consecutive frames; however, it is applicable only to moving-view scenarios, whereas our method does not rely on this assumption. Moreover, in these methods, the overhead of sorting and the possibility of replacing it with a more efficient alternative remain insufficiently explored. A comparison with related 3DGS accelerators is presented in Table I. These limitations motivate a more in-depth analysis of rasterization and sorting, culminating in the two key challenges outlined below:

- **Challenge-1: Computational redundancy in rasterization.** Most existing implementations adopt a pixel-wise mapping strategy, where each pixel is rasterized independently by a separate processing element (PE) in an accelerator or a

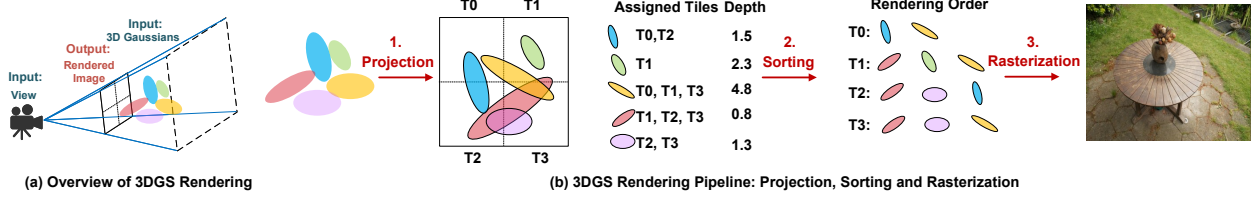


Fig. 1. The rendering process of 3D Gaussian Splatting.

CUDA core in a GPU. While this approach offers high parallelism, it ignores the substantial overlap in intermediate computations shared across neighboring pixels. As a result, many common terms are redundantly recalculated along rows and columns, leading to unnecessary multiply-and-accumulate (MAC) operations. This redundancy significantly increases latency or PE overhead, making rasterization inefficient. A deeper analysis is provided in Sec. II-B1.

- **Challenge-2: Sorting scalability and pipeline imbalance.** Hardware implementations of sorting often rely on parallel sorting modules [20], whose area and cost scale rapidly with input parallelism. Moreover, sorting and rasterization have inherently different computational complexities, $O(N \log^2 N)$ for sorting versus $O(N)$ for rasterization, posing challenges for maintaining pipeline balance. This imbalance is further exacerbated by large variations in the number of Gaussians per tile, which span up to two orders of magnitude in our profiling. Consequently, fixed-parallelism sorting modules either lead to resource underutilization on small workloads or cause rasterization bottlenecks for large workloads. A detailed analysis is provided in Sec. II-B2.

To address **Challenge-1**, we propose an *axis-shared rasterization* technique that eliminates redundant computations within each tile. The core idea is to compute common intermediate terms along the X- and Y-axes only once and then share these terms across all processing elements (PEs) within the tile. Each PE can then complete rasterization by simply combining the shared terms. This design reduces the number of MAC operations by 38% while preserving high parallelism. As a result, it enables either lower latency under the same PE resources or reduced PE overhead with equivalent latency. Building on this principle, we further design a highly efficient PE array architecture for rasterization.

To address **Challenge-2**, we adopt a hardware-algorithm co-design approach. On the algorithmic side, we re-examine the role of sorting in Gaussian Splatting, which traditionally enforces depth ordering for transmittance computation and ultimately determines the decay factor for color blending. Our key insight is that the decay factor can be computed directly without explicit sorting. By recognizing the analogy between 3DGS and image composition [40], and inspired by order-independent transparency techniques [2], [30], we develop a novel *order-independent transmittance* (OIT) method tailored to 3DGS. This method, co-designed with the hardware, leverages a lightweight MLP to predict decay factors, mitigating workload imbalance while preserving image quality. On the hardware side, we introduce a unified and reconfigurable PE

array that supports both rasterization and MLP inference, thereby removing the need for a costly sorting engine and sustaining consistently high utilization.

In conclusion, we make the following contributions:

- We identify two previously overlooked challenges in accelerating 3D Gaussian Splatting, (i) computational redundancy in rasterization and (ii) scalability and pipeline imbalance issues caused by sorting (Sec. II-B).
- We propose an *axis-shared rasterization* technique and a dedicated hardware design, eliminating redundant computations and reducing MAC operations by 38% (Sec. III).
- We develop a novel *order-independent transmittance* method that bypasses explicit sorting and enables efficient decay factor prediction with negligible quality loss (Sec. IV).
- We design a *unified and reconfigurable hardware accelerator* that supports both rasterization and MLP inference, achieving real-time 3DGS rendering with consistently high utilization (Sec. V).

II. BACKGROUND AND MOTIVATION

A. 3D Gaussian Splatting

3DGS parameters. 3DGS represents a 3D scene as a collection of Gaussians. For a given camera view, an image is rendered by splatting these Gaussians into 2D space following the standard 3DGS pipeline. Each Gaussian is defined by Equation (1), where p' denotes the 3D coordinates. In total, 59 parameters are required to describe each Gaussian: i) the mean (position) μ' (3 parameters); ii) the size and shape, represented by the covariance matrix Σ' , which is determined by the scale s (3 parameters) and rotation q (4 parameters); iii) the opacity factor o (1 parameter); and iv) the view-dependent color, parameterized by spherical harmonics (SH) coefficients ($16 \times 3 = 48$ parameters).

Rendering steps. The overall rendering process takes the camera parameters, pose, and 3D Gaussians as input, and synthesizes a final image as output, as shown in Fig. 1(a). The rendering pipeline consists of three main steps—projection, sorting, and rasterization—as illustrated in Fig. 1(b). For *projection*, based on the camera parameters and pose, 3D Gaussians are projected into 2D Gaussians. Specifically, the 3D mean (μ') and covariance (Σ') are projected into a 2D mean μ (2×1) and a 2D covariance Σ (2×2). The depth (d) of each 3D Gaussian relative to the camera is also calculated (e.g., 0.8–4.8 in the figure). In addition to spatial attributes, color is represented by an RGB vector (3×1), computed from the SH coefficients and camera pose. Rendering is performed

at a fixed tile granularity of 16×16 pixels. After projection, each 2D Gaussian is mapped to the tiles it overlaps. For example, in the toy case shown, the blue Gaussian overlaps tiles T_0 and T_1 , while the green Gaussian overlaps only T_1 . For *sorting*, since the relative depth order determines occlusion and affects α -blending, each tile sorts its intersecting Gaussians in ascending depth order (near to far). As shown in Fig. 1, the correct order for tile T_0 is the blue Gaussian followed by the yellow. For *rasterization*, the pipeline computes each Gaussian’s contribution to every pixel within a tile, ultimately synthesizing the final image. This process consists of α -computation followed by α -blending. The α value is computed using Equation (2), where $\mathbf{p}$ (2×1) denotes the pixel position, μ and Σ are the projected 2D Gaussian mean and covariance, and o is the opacity factor. Based on the computed α values, α -blending determines the final pixel color C as defined in Equation (3) (left), where T_i denotes the accumulated transmittance, i indexes the sorted Gaussians, and c_i denotes each Gaussian’s RGB color. The transmittance T_i is computed from all $i - 1$ preceding Gaussians, as given in Equation (3) (right).

$$G(\mathbf{p}') = e^{-\frac{1}{2}(\mathbf{p}' - \mu')^T \Sigma'^{-1}(\mathbf{p}' - \mu')}, \quad (1)$$

$$\alpha = o \cdot e^{-\frac{1}{2}(\mathbf{p} - \mu)^T \Sigma^{-1}(\mathbf{p} - \mu)}, \quad (2)$$

$$C = \sum_{i=1}^N T_i \alpha_i c_i, \quad T_i = \prod_{j=1}^{i-1} (1 - \alpha_j) \quad (3)$$

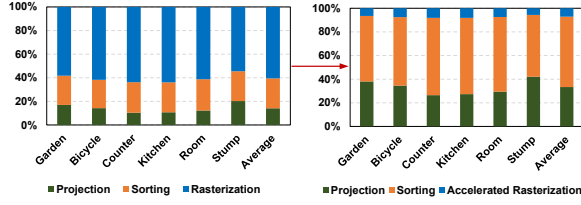


Fig. 2. 3DGS latency breakdown on GPU (left) and GPU with hardware accelerated rasterization(right).

Profiling and analysis. We conducted a detailed profiling of the three rendering steps on the NVIDIA Jetson Orin Nano GPU using the MipNeRF-360 dataset [1], as shown in Fig. 2 (left). The results indicate that projection, sorting, and rasterization account for 14.2%, 25.3%, and 60.5% of the total latency, respectively. Since sorting and rasterization together constitute nearly 90% of the latency, our work primarily targets the acceleration of these two steps. An analysis of the MAC count per Gaussian within the rasterization step shows that α -computation requires 8 multiplications, 4 additions, and 1 exponential operation, whereas α -blending requires 5 multiplications and 4 additions. This reveals that α -computation is the most MAC-intensive operation, motivating our focus on optimizing it.

B. Challenge Analysis and Motivation

1) **Challenge-1: Computational redundancy in rasterization.** As demonstrated in profiling, rasterization is the most time-consuming component in 3DGS. Within the rasterization

pipeline, α -computation is the most MAC-intensive process, involving cascaded matrix-vector multiplications. The formula expansion for α computing is shown in Fig. 3 (top), where μ_i^x and μ_i^y denote the center of the i -th Gaussian, x and y denote the coordinates of a pixel. The conic matrix (Σ^{-1}), which is defined as the inverse of 2D Gaussian’s covariance matrix, is parameterized by a_i , b_i , and c_i . According to the formulation, it requires 8 multiplications (MUL), 4 additions (ADD), and 1 exponential operation (EXP). Conventionally, prior work [25] designs a PE array for rasterization, where the computation for each pixel is mapped to one PE. In these designs, the PE structure reflects the theoretical MAC count derived from the α computation formulation, as shown in Fig. 3 (bottom), with registers omitted for simplicity.

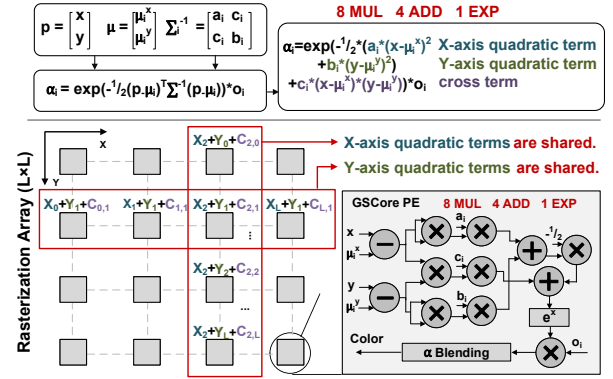


Fig. 3. The sources of redundant computing in rasterization.

To illustrate the source of redundancy, we decompose the exponent in the α computation into three components, as shown in Fig. 3 (top): (i) the X-axis quadratic term, representing the squared distance between the pixel and the Gaussian center along the X-axis; (ii) the Y-axis quadratic term, representing the squared distance along the Y-axis; and (iii) the cross term, capturing the interaction between the X and Y coordinates. When mapping pixel computations onto the PEs of the rasterization array (Fig. 3, bottom), clear spatial redundancy emerges: within any row of pixels, the Y-axis quadratic terms remain identical across PEs, while within any column, the X-axis quadratic terms are shared. This analysis highlights substantial redundancy in both the X- and Y-axis quadratic terms. Yet conventional PE designs recompute these terms independently for every pixel, missing the opportunity to exploit this redundancy for computational savings, as illustrated in Fig. 3 (bottom right).

To avoid redundant computation, our key idea is to redesign the entire dataflow by precomputing the axis-shared terms. Each PE then performs only a simple combination of these precomputed values, thereby significantly reducing complexity while maintaining parallelism. This axis-shared rasterization is implemented through a dedicated hardware architecture (Sec. III) to improve the compute efficiency.

2) **Challenge-2: Sorting scalability and pipeline imbalance issues.** According to Amdahl’s Law [41], if only rasterization is accelerated while other steps remain on the

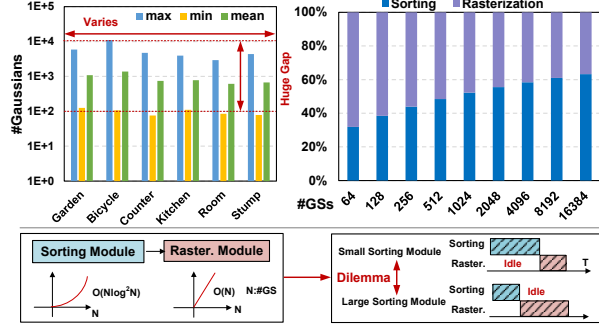


Fig. 4. The number of Gaussians varies across tiles and scenes and leads to pipeline imbalance issue.

GPU, sorting inevitably becomes the primary bottleneck, as shown in Fig. 2 (right). To quantitatively assess the impact of sorting on per-tile rendering, we profiled the MipNeRF-360 dataset [1]. We employed a trained Gaussian checkpoint at 7k iterations and used the validation set to measure the variation in Gaussian counts across tiles and scenes. As shown in Fig. 4 (top left), the per-tile Gaussian count ranges from approximately 80 to over 10,000, spanning more than two orders of magnitude. This variation occurs both within tiles of a single scene and across different scenes, presenting a fundamental challenge for efficient hardware design.

We further profile the latency of rasterization and sorting across different Gaussian counts per tile, following the hardware architecture of [25], as shown in Fig. 4 (top right). When the Gaussian count is small, the sorting latency can be less than half that of rasterization, whereas for large counts, it can grow to nearly twice as high. This trend can be explained through complexity analysis. Let N denote the number of Gaussians mapped to a tile. Sorting typically incurs $O(N \log^2 N)$ complexity, as in bitonic sort [20], with hardware area scaling as $k \log^2 k$ under input parallelism k , leading to rapidly increasing overhead as k grows. In contrast, rasterization is a MAC-dominated operation with linear complexity $O(N)$. Due to this heterogeneity, sorting and rasterization cannot be efficiently unified into a single engine. Moreover, the disparity in computational complexity, combined with the large variance in Gaussian counts across tiles, inherently causes pipeline imbalance. As illustrated in the schedule of Fig. 4 (bottom), under a fixed area budget, a small sorting module becomes the bottleneck for tiles with many Gaussians, stalling rasterization and reducing utilization. Conversely, a larger sorting module with higher overhead shifts the bottleneck to rasterization for tiles with fewer Gaussians. Thus, designing an architecture that achieves balanced performance across diverse scenes and tile workloads remains a significant challenge.

To comprehensively address this challenge, we adopt an algorithm–hardware co-design strategy. At the algorithmic level, we revisit the role of sorting and entirely replace it with an order-independent transmittance method (Sec. IV). It exploits the high throughput of the PE array through reconfigurability, enabling uniform support for both transmittance computation

and rasterization (Sec. V). Consequently, regardless of the number of Gaussians per tile, the unified hardware sustains consistently high utilization.

III. AXIS-SHARED RASTERIZATION

Inspiration and overview. We propose axis-shared rasterization to address *Challenge-1* (Sec. II-B1). The approach involves three stages: 1) computing shared terms along the X and Y axes, 2) broadcasting them to each PE, and 3) combining them in each PE to obtain the final result. To efficiently implement axis-shared rasterization, our key considerations involve (i) simplifying the control logic of the three-stage process, and (ii) minimizing or eliminating the additional storage required. For a tile of size $L \times L$, shared-term computation has $O(L)$ complexity, while the combination stage has $O(L^2)$ complexity, with each shared term reused L times. This observation motivates the use of an $L \times L$ array for the combination stage, supported by L -sized preprocessing modules for shared-term computation. Following these principles, Fig. 5 (top left) shows the design overview. For a 16×16 tile, the design includes: (i) a 16×16 rasterization array, (ii) a 16-element X-PE line, and (iii) a 16-element Y-PE line. The X-PE line computes X-axis shared terms, while the Y-PE line computes Y-axis shared terms. Each X-PE and each Y-PE broadcast their results to the corresponding rasterization PEs in the same row or column, repeated 16 times. The PE size is determined by the algorithm [22], and this broadcast overhead is small enough to implement and meet timing closure requirements.

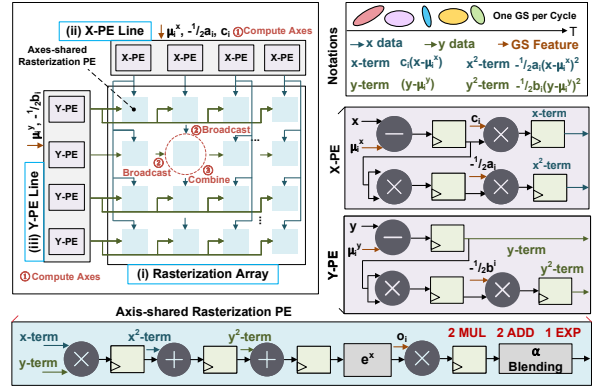


Fig. 5. Hardware and computation flow of axis-shared rasterization.

Computation flow and PE structure. The rasterization array renders each Gaussian and computes its output to a 16×16 tile of pixels continuously in each cycle. The Gaussian parameters are provided as input to the X-PE line and Y-PE line, which perform the axes computation, and the outputs from these PE lines are then fed to the rasterization PE array. To eliminate redundant processing of the Gaussian conic matrix parameters (e.g., computing the factor $-\frac{1}{2}$), we directly store the parameter set $\{-\frac{1}{2}a_i, -\frac{1}{2}b_i, c_i\}$, where i denotes the index of each Gaussian.

The X-PE line receives the Gaussian parameters c_i and $-\frac{1}{2}a_i$, which are broadcast to all X-PEs. Internally, each X-

PE line generates 16 x-coordinates ($x_0 \sim x_{15}$), which are automatically incremented by 16 when shifting to the next tile on the right. With notations shown in Fig. 5 (top right), the detailed X-PE structure is illustrated in Fig. 5 (right). For simplicity, the x index is omitted (similarly for y in the following description). Each X-PE consists of one adder and three multipliers. One computation branch calculates the term from X-axis (x -term) by computing $(x - \mu_i^x)$ and multiplying it by c_i , while the other branch squares $(x - \mu_i^x)$ and multiplies it by $-\frac{1}{2}a_i$ to obtain the X-axis quadratic term (x^2 -term). The Y-PE line receives the Gaussian parameter $-\frac{1}{2}b_i$, which is broadcast to each Y-PE. The y-coordinates ($y_0 \sim y_{15}$) are similarly incremented by 16 when moving vertically between tiles. As shown in Fig. 5 (right), each Y-PE contains one adder and two multipliers. One computation branch produces the Y-axis term (y -term) by computing $(y - \mu_i^y)$ directly, while the other squares $(y - \mu_i^y)$ and multiplies it by $-\frac{1}{2}b_i$ to generate the Y-axis quadratic term (y^2 -term).

For the *rasterization PE array*, each PE receives vertical inputs from the corresponding X-PE and horizontal inputs from the corresponding Y-PE based on the pixel coordinates. As shown in Fig. 5 (bottom), each rasterization PE comprises only one adder and three multipliers. One multiplier is dedicated to multiplying the opacity factor o_i at the final stage, while the remaining MAC unit combines the inputs from the PE lines. Specifically, the multiplier computes the product $x \times y$, and the two adders subsequently add this result to the x^2 -term and y^2 -term, respectively. *Considering synchronization*, the X-PE line initiates one cycle earlier than the Y-PE line, ensuring that the x term and y term arrive simultaneously at each rasterization PE during the first cycle, with the x^2 -term and y^2 -term arriving in the second and third cycles, respectively. After executing the exponential operation and multiplying by o_i , the computation of α^i is completed, and the result is then fed into the α -blending unit. Due to the continuous processing of Gaussians, the computation of α^{i+1} is completed in the subsequent cycle.

Overhead analysis. Compared to the GScore implementation, our axis-shared rasterization requires an additional X-PE line and Y-PE line while significantly simplifying the design of the rasterization PE. To ensure a fair comparison, the MAC cost of the extra PE lines is amortized across the rasterization PE array. *For α -computation*, the total number of multipliers is calculated as $16 \times (3 + 2) = 80$ for the X-PE and Y-PE lines and $16 \times 16 \times 2 = 512$ for the rasterization PE array, resulting in a total of 592. Similarly, the total number of adders is calculated as $2 \times 16 = 32$ for the extra PE lines and $16 \times 16 \times 2 = 512$ for the rasterization PE array, resulting in a total of 544. When averaged over a 16×16 rasterization PE array, our axis-shared approach requires only 2.31 multipliers and 2.13 adders per PE. Compared with 8 multipliers and 4 adders in the GScore implementation, the counts are reduced by 63% for the α -computation stage. *For the complete rasterization process including α -blending*, the GScore design requires 12 multipliers and 8 adders per PE. In contrast, our design requires only 6.31 multipliers and 6.13 adders per PE after amortization. This corresponds to

an overall MAC reduction of approximately 38%.

IV. MLP-BASED ORDER-INDEPENDENT TRANSMITTANCE

A. Algorithmic Motivation

The sorting process is challenging for on-chip deployment, as analyzed in *Challenge-2* (Sec. II-B2). But it is essential for the original 3DGS due to α -blending, which depends critically on the relative depth order determined by the camera pose. However, reexamining the blending Equation (3), we observe that the ultimate objective of sorting is to compute the correct transmittance T_i . The Gaussians preceding the i th Gaussian have smaller depth, and each contributes a factor of $1 - \alpha_j$ (for $j = 0, 1, \dots, i-1$) in the cumulative product. Since each α lies in the interval $(0, 1)$, as the depth of the i th Gaussian increases, the transmittance T_i decreases, thereby serving as a decay factor. This naturally raises the question: *Can we directly compute the decay factor based on depth?*

Inspiration from image composition. The 3DGS paper [22] demonstrates that its α -blending adopts the NeRF-style volumetric model, while reusing the classic graphics term [40]. “ α ” stems from the interpolation $\alpha A + (1 - \alpha)B$ [42], corresponding to the “*over*” operation of A over B . Thus, 3DGS blending is analogous to image composition. Fig. 6 illustrates a three-Gaussian example: 3DGS blends front-to-back, while image compositing is back-to-front. In this context, C_3 , C_2 , and C_1 represent accumulated colors obtained through successive *over* operations with colors and opacities (c, α) , yielding results identical to 3DGS. This equivalence readily generalizes to any number of Gaussians.

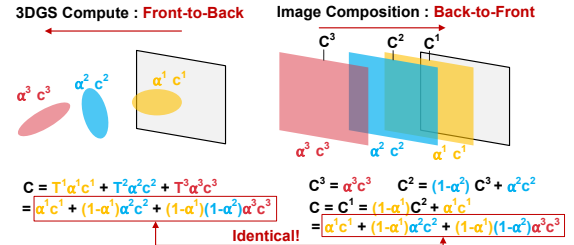


Fig. 6. 3DGS α -blending is analogous to image composition.

Because the “*over*” operation is non-commutative, depth sorting is computationally expensive. To address this, several order-independent transparency (OIT) techniques have been developed in computer graphics [2], [5], [6], [30], [34]. Among them, weighted OIT assigns weights via a monotonic decreasing function of depth $F(d_i)$ and then accumulates colors C accordingly, producing images with negligible quality loss, as formulated in Equation (4). This success, together with the conceptual similarity between image composition and 3DGS α -blending, motivates our exploration of order-independent transmittance (also abbreviated as OIT) for 3DGS.

$$C = \frac{\sum_{i=1}^n F(d_i) \alpha_i c_i}{\sum_{i=1}^n F(d_i) \alpha_i} \quad (4)$$

$$C = \frac{\sum_{i=1}^n F(d_i, x, y, z) \alpha_i c_i}{\sum_{i=1}^n F(d_i, x, y, z) \alpha_i} \quad (5)$$

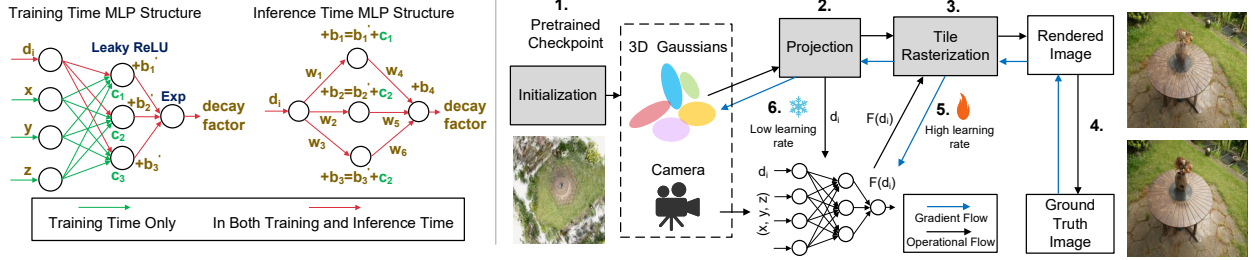


Fig. 7. Neural network structure (left) and the training framework (right).

Variables and form of function. However, weighted OIT relies on manually selected and tuned decay functions $F(d_i)$. Recent work [18] demonstrates that sorting in 3DGS can be approximated by a fixed set of functions. However, their approach is restricted to handcrafted depth-based functions, which limits their ability to model complex dependencies and generalize across diverse scenes. In contrast, we observe that a defining characteristic of 3DGS is its view-dependent synthesis, where camera view critically influences rendering. For different views, even identical Gaussian depths may contribute differently to the final color, and our experiments will further demonstrate the significance of view information in Sec. VI-B. To capture this effect, we integrate view information into the input by extending Equation (4) into Equation (5), where (x, y, z) denotes the normalized view direction vector. This vector, determined only by the camera pose, captures the global viewing ray orientation and is independent of individual Gaussians. Crucially, the interplay between depth and view direction is highly nonlinear and difficult to capture with fixed-form functions. In contrast, a Multi-layer Perceptron (MLP), leveraging its universal regression capability, can model these joint dependencies with high fidelity. Since 3DGS supports rapid per-scene training on desktop or server GPUs, jointly training a small MLP is both practical and computationally efficient, requiring around 30 minutes per scene according to our experiment (Sec. VI-A).

B. MLP-based OIT Design and Training Pipeline

Hardware-aware MLP design. The MLP design decides the trade-off between algorithmic expressiveness and hardware performance. From a hardware perspective, we aim to reuse the highly parallel MAC units in the rasterization array to accelerate MLP computation, thereby constraining its structure. As shown in Fig. 7 (left), the MLP adopts distinct structures for training and inference: green components are active only in training, while red components remain in both phases. During training, the MLP takes as input the Gaussian depth d_i and the view direction (x, y, z) , using ground-truth images to learn the decay function. During inference, the view direction is constant across Gaussians for a given camera pose and can therefore be precomputed once. Specifically, its contribution is fused into the MLP’s bias terms: the precomputed values c_1, c_2, c_3 are incorporated into the original biases b'_1, b'_2, b'_3 , yielding updated biases b_1, b_2, b_3 . Leaky ReLU [49] with a coefficient of $\frac{1}{8}$ serves as the first-layer activation, and

an exponential function is used in the output layer. The exponential function is selected for its simplicity and wide dynamic range, while also reusing the exponential unit already present in rasterization. The resulting inference-time MLP is lightweight, containing only 2 layers and 10 parameters, and requiring just 6 MACs, well suited for efficient on-chip deployment.

Training pipeline. Our training framework is illustrated in Fig. 7 (right), where black arrows indicate the forward pass and blue arrows denote gradient flow during backpropagation. We initialize the 3D Gaussians with a pre-trained checkpoint from the original sorting-based algorithm (denoted by 1), which significantly reduces training cost, since co-training Gaussians and the MLP from scratch requires small learning rates and converges slowly. After initialization, the Gaussians are projected into 2D space given the camera pose (denoted by 2), and their depths are extracted as inputs d_i to the MLP. With sorting eliminated, the MLP outputs $F(d_i)$ enable direct tile rasterization (denoted by 3), and the rendered image is obtained via Equation (5). The rendered result is then compared against the ground truth to compute the loss (denoted by 4), following the original setting [22]. As shown on the rightmost side, early training fails to capture occlusion, producing partially transparent objects. As training progresses, however, the rendered images steadily improve and converge toward the ground truth.

The backward process follows the blue paths, where gradients optimize both the 3D Gaussians and the MLP. Their learning rates, however, differ substantially. We apply a scaling factor (e.g., 0.01) to the original 3DGS learning rate, while assigning a larger learning rate to the MLP. This enables the loss to quickly optimize the MLP (denoted by 5) while refining the 3D Gaussians more gradually (denoted by 6). Another key design choice is to disable Gaussian cloning and splitting, as these operations introduce abrupt changes that undermine training stability. Keeping the number of Gaussians constant produces more stable and accurate rendering results. Overall, this framework integrates order-independent transmittance learning into 3DGS, achieving efficient training without explicit depth sorting.

V. ARCHITECTURE DESIGN AND OPTIMIZATION

A. Reconfigurable Hardware Design

Overview. Fig. 8 (left) illustrates the unified hardware architecture supporting both rasterization and MLP computation,

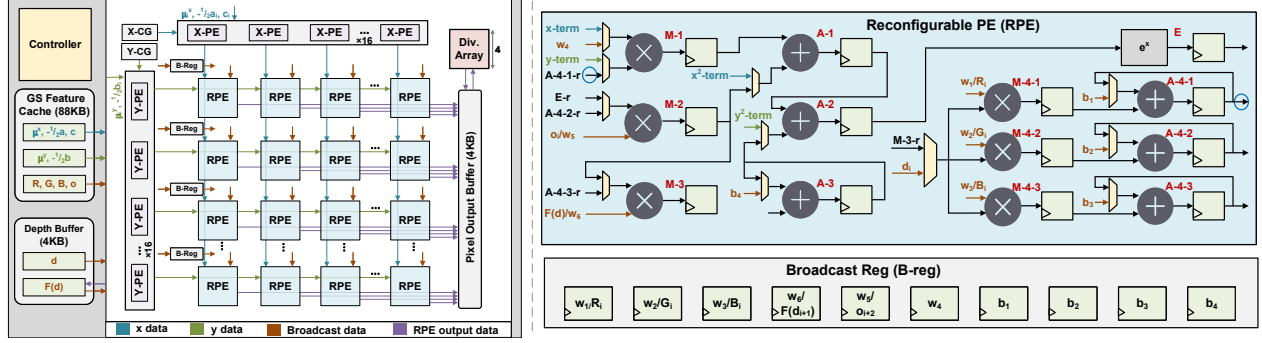


Fig. 8. Unified hardware architecture of rasterization and MLP inference (left), and the structure of modules (right).

with arrow notation shown at the bottom left. The architecture consists of two main components: dedicated computation and storage modules. The computation unit features a 16×16 reconfigurable PE array with per-row broadcast registers, along with X-PE and Y-PE lines (introduced in Sec. III). Both PE lines receive x or y coordinates from the coordinate generator (CG), which conducts the tile schedule across the image. For storage, projected Gaussian (GS) features are placed in a feature cache (9 parameters per GS), whereas depth values and their decay factors $F(d_i)$ are stored in a separate depth buffer to account for differing on-chip bandwidth demands. Finally, the numerator and denominator from Equation (5) are written to the pixel output buffer and normalized by the division array to produce the final pixel values.

Reconfigurable PE. Fig. 8 (right) shows the reconfigurable PE, consisting of 6 multipliers, 6 adders, and one exponential unit, all in FP16 precision. The multipliers and adders (MACs) are organized into two groups of three. In the first group, each MAC operates independently ($M\text{-}\{1 \sim 3\}$, $A\text{-}\{1 \sim 3\}$), while in the second group they operate cooperatively ($M\text{-}4\{1 \sim 3\}$, $A\text{-}4\{1 \sim 3\}$). Each MAC is paired with a register to ensure timing closure at high frequency. Some connections are denoted implicitly: the suffix $-r$ marks a register output. For example, $A\text{-}4\text{-}1\text{-}r$ (fourth wire on the left, blue circle) connects to the register output of the $A\text{-}4\text{-}1$ adder (second wire on the right, blue circle) in the same PE. Multiplexer control selects the datapath configuration, switching between rasterization mode (upper) and MLP mode (lower). The workflow and corresponding PE configurations are as follows:

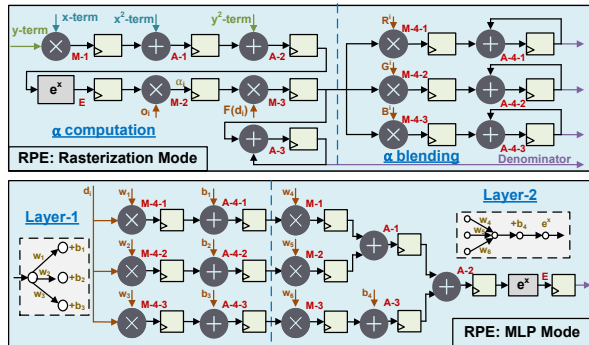


Fig. 9. Reconfigurable PE for rasterization and MLP modes.

Rasterization mode. In rasterization mode, the PE is configured as shown in Fig. 9 (top). The PE array processes one GS per cycle for the 16×16 tile pixels. The GS-feature buffer supplies GS position (μ_i^x, μ_i^y) and conic parameters ($-\frac{1}{2}a_i, -\frac{1}{2}b_i, c_i$) to the X-PE and Y-PE lines, where i is the GS index. The unique o_i is loaded into the broadcast register and broadcast 16 times to all PEs on the same line. Within each PE, $M\text{-}\{1 \sim 2\}$, $A\text{-}\{1 \sim 2\}$, and the exponential unit perform the rasterization, as shown in Fig. 5. The X-PE, Y-PE structure and α computation workflow are described in Sec. III. The additional step is α -blending, defined in Equation (5):

GS color features (R_i, G_i, B_i) are loaded into the broadcast register, which has 10 units, 5 of which are used in rasterization mode (Fig. 8, right bottom). The decay factor $F(d_i)$, sourced from the depth buffer, is also loaded into the broadcast register. After broadcasting R_i, G_i, B_i , and $F(d_i)$ 16 times to each line, the PEs compute Equation (5). Specifically, $M\text{-}3$ multiplies $F(d_i)$ with α_i , and $A\text{-}3$ accumulates $F(d_i)\alpha_i$ across Gaussians to form the denominator. Meanwhile, $M\text{-}4\{1 \sim 3\}$ multiplies $F(d_i)\alpha_i$ with R_i, G_i, B_i , and $A\text{-}4\{1 \sim 3\}$ accumulates the results across GSs for the RGB channels.

MLP mode. The MLP mode configuration of the PE is illustrated in Fig. 9 (bottom). In this mode, the PE array processes $16 \times 16 = 256$ depth values per cycle, producing 256 corresponding $F(d)$ outputs. Inputs are read from the depth buffer with high on-chip bandwidth, and the results are written back to the same buffer. The MLP weights reside in the broadcast register, fully utilizing all ten units. As shown, $M\text{-}4\{1 \sim 3\}$ and $A\text{-}4\{1 \sim 3\}$ compute the first layer, including bias addition. The second layer is handled by $M\text{-}\{1 \sim 3\}$, with $A\text{-}\{1 \sim 3\}$ performing accumulation and bias addition. The exponential unit is reused to implement the exponential activation function. For simplicity, Leaky ReLU is omitted from the figure. It is implemented via sign detection and a 5-bit integer adder, subtracting 3 from the FP16 exponent when the input is a normal negative. For subnormal negative or positive values, the input remains unchanged.

B. Fine-grained Interleaved Pipeline

Memory-bound issue. Since our MLP inference only relies on the depth of each GS, irrelevant to the tile assignment, it is straightforward to compute the $F(d)$ for the entire GSs, followed by tile-by-tile rasterization, as shown in the naive

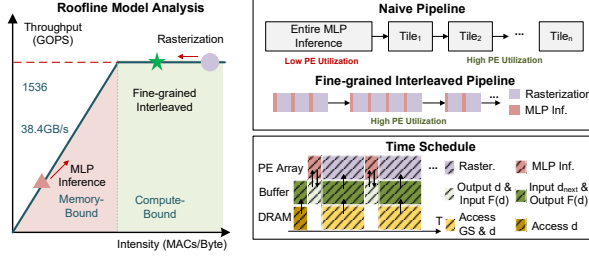


Fig. 10. Roofline model analysis and pipeline comparison.

pipeline of Fig. 10 (top right). However, we observe that PE utilization is extremely low during MLP inference. The reason for this is that the operational intensity of MLP inference and rasterization differs significantly. For rasterization, each projected Gaussian comprises 9 parameters and performs 256×6 MAC operations. In contrast, for MLP inference, 1 depth parameter only incurs 6 MAC operations, indicating a nearly 30-fold difference. Given a typical configuration for our architecture, as shown in Fig. 10 (left), rasterization is compute-bound, whereas MLP inference is heavily memory-bound [47]. This memory-bound issue also makes deploying our optimized order-independent transmittance algorithm directly to GPUs not an optimal choice.

Optimization. We propose a fine-grained interleaved pipeline that subdivides each tile into subtiles, as shown in Fig. 10 (right). The key idea is to overlap rasterization of the current subtile with memory access for MLP inference of the next. Fine-grained subtile processing is required because tiles contain variable numbers of GS while the depth buffer capacity is limited. Fig. 10 (bottom right) shows how the pipeline maps onto hardware. Starting with the first subtile, DRAM transfers GS depths d to the depth buffer, which then forwards them to the PE array for MLP inference. The resulting $F(d)$ values are written back to the depth buffer. The PE array then rasterizes using $F(d)$ from the depth buffer, while the buffer simultaneously loads depths for the next subtile, overlapping rasterization with memory access. This process repeats, fully hiding depth-access latency except for the first subtile.

C. Off-chip Access Optimization

Tile schedule. Our architecture adopts a *GS-feature* cache, where each cache line is tagged using a 28-bit GS ID, and an additional 4 bits record the number of tiles intersected by each GS. The cache prioritizes replacing less important GSs. These 4 bits, together with the 28-bit ID, form a 32-bit aligned storage entry. This design effectively reduces off-chip accesses by leveraging the spatial locality of GSs. However, different tile scheduling trajectories influence cache hit rates. As shown in Fig. 11(a), the baseline implementation scans tiles in a row-by-row manner, exploiting horizontal locality but lacking vertical and hierarchical locality. For instance, it can reuse GSs intersecting horizontally aligned tiles (e.g., 1×2 , 1×3), but not vertically aligned ones (e.g., 2×1 , 3×1). A slight improvement is achieved using the S-trajectory, which reverses direction at the end of each row.

Inspired by Morton encoding [33], which interleaves the y - and x -axis bits (e.g., interleaving y_1y_0 with x_1x_0 to obtain $y_1x_1y_0x_0$), we schedule tiles in increasing Morton code order, forming a Z-trajectory that better preserves 2D locality, as shown in Fig. 11(b). However, the diagonal segments of this trajectory may span large spatial distances, causing discontinuities. To mitigate this issue, and inspired by the continuity of Gray code [13], we modify the Z-trajectory into a “ π ” trajectory with improved continuity, as shown in Fig. 11(c). Both Z- and π -trajectories exhibit hierarchical locality, as indicated by the blue arrow (trajectory of 2×2 tiles), making them inherently scalable. This curve corresponds to the Hilbert curve [17], originally proposed for fractal geometry in 1891. We further generalize the design for different 3DGS image resolutions, as shown in Fig. 11(d). Specifically, the π -trajectory is applied only within each 8×8 tile block, while block-level traversal follows the S-trajectory. For images where the tile count is not divisible by 8, the remaining tiles are scheduled using a row-wise S-trajectory.

VI. EVALUATION

A. Experimental Methodology

Algorithm Setup. Datasets and baselines: Our implementation and GPU-based inference are built on `Gsplat`, a widely used and efficient 3DGS library [51]. Following [22], we evaluate on real-world scenes from the MipNeRF-360 dataset [1], including *garden*, *bicycle*, *stump*, *bonsai*, *counter*, *kitchen*, and *room*. We compare our order-independent transmittance method with the sort-free weight-sum 3DGS algorithm [18]. **Evaluation metrics:** Rendering quality is evaluated using three standard metrics: Peak Signal-to-Noise Ratio (PSNR, higher is better), Structural Similarity Index (SSIM, higher is better), and Learned Perceptual Image Patch Similarity (LPIPS, lower is better). **Implementation details:** Training is performed on an NVIDIA RTX 3090 GPU, with checkpoints obtained after 7000 epochs following [22]. The model is then trained for an additional 10000 epochs, initialized from these checkpoints. The MLP learning rate is set to 0.005, while Gaussian learning rates are scaled by 0.01. Thanks to its lightweight design and the convenient training framework, training is highly efficient, requiring around 30 minutes per scene on an RTX 3090 GPU. Moreover, the model is trained offline only once and can be reused across various edge deployments.

Hardware Setup. Our architecture is implemented in SystemVerilog and synthesized with Design Compiler using the TSMC 28nm CMOS library. Rendering is performed with FP16 arithmetic based on DesignWare IP [43]. The design is fully pipelined and operates at 1 GHz. A DDR5-4800 DRAM with 38.4 GB/s bandwidth is modeled using Ramulator [24], while on-chip SRAM energy and area are estimated with CACTI [35]. Total energy consumption, including both on-chip and off-chip memory accesses, is obtained using DRAMPower [3]. Latency and memory traffic are evaluated with a cycle-accurate simulator, cross-validated against RTL simulation results. Table II summarizes the design metrics, and our design occupies 3.85 mm^2 and consumes 1.64 W.

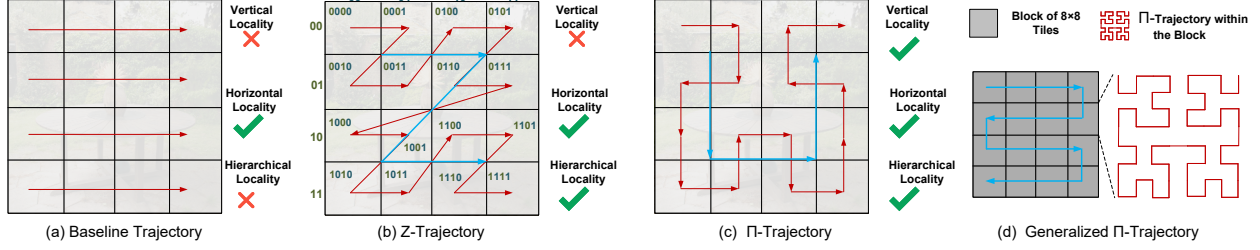


Fig. 11. Comparison of different tile schedule trajectories with 4×4 size as the example.

Baselines. We benchmark against the NVIDIA Jetson Orin Nano edge GPU [38] and the desktop-class NVIDIA RTX 3090, demonstrating substantially lower area and power while achieving higher performance, as detailed in Sec. VI-D. We further compare against the state-of-the-art 3DGS accelerators GScore, MetaSapiens, and GBU [25], [29], [52].

TABLE II
AREA AND POWER OF OUR DESIGN.

Component	Configuration	Area [mm ²]	Power [W]
Reconfigurable PE Array	16×16 Reconfigurable PE X-PE Line + Y-PE line +	2.958	1.48
Support Modules	Coord. Gen. + Div. Array (4 Div.)	0.064	0.02
On-chip Buffer	GS Feature (88KB) + Output (4KB) + Depth (4KB)	0.826	0.14
Total		3.85	1.64

B. Algorithm Accuracy Evaluation

Render quality comparison. Our axis-shared rasterization is numerically identical to the original implementation; therefore, rendering quality differences arise solely from the MLP-based order-independent transmittance (OIT). Table IV reports the rendering quality of our MLP-based OIT and compares it with (i) the original 3DGS baseline and (ii) the state-of-the-art sort-free weight-sum rendering method [18]. The weight-sum method in [18] proposes several depth-based weighting functions; we report the best-performing variant, LC-WSR (without view-dependent opacity), for fair comparison. To further analyze the role of view information, we evaluate an ablation version of our model using depth-only input (denoted as OIT+d), while the full model incorporating both depth and view direction is denoted as OIT+d+view. As shown in Table IV, incorporating view information yields consistent improvements. PSNR increases from 26.17 (OIT+d) to 26.90 (OIT+d+view), accompanied by higher SSIM and lower LPIPS. Overall, our full model achieves a PSNR of 26.90, with only a minor 0.3 degradation compared to the original sorted baseline. SSIM remains nearly unchanged, and LPIPS is slightly improved. Compared with prior weight-sum rendering [18], our approach consistently achieves better results across all three evaluation metrics. Fig. 12 visually compares our method with weight-sum rendering [18] and original 3DGS, as highlighted by the blue boxes, while the weight-sum method exhibits slight blending artifacts in these regions, our method better preserves local occlusion relationships and depth layering, producing results that more closely match the original 3DGS.

Methodological comparison with related works. Weight-sum rendering [18] adopts a monotonic depth-based weighting

TABLE III
OVERHEAD COMPARISON BETWEEN GPUS AND OUR DESIGN.

Device	Tech.	Area	Power	On-chip SRAM	DRAM Bandwidth	Cores
Jetson Orin Nano	8 nm	200	~15 W	~3 MB (L1+L2)	68.2 GB/s	1024 CUDA
RTX 3090	8 nm	628	350 W	6 + 10.25 MB	936 GB/s	10496 CUDA
Ours	28 nm	3.85	1.64 W	96 KB	38.4 GB/s	256 PEs

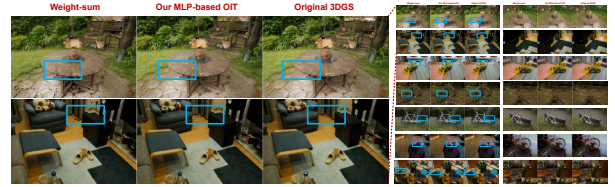


Fig. 12. Visual comparison among sort-free methods.

function. While computationally lightweight, it remains a handcrafted approximation that cannot fully capture complex Gaussian interactions in anisotropic 3DGS scenes. In contrast, our MLP-based OIT provides a data-driven representation. The MLP offers greater expressive capacity than prior methods and flexibly learns an optimized mapping for transmittance estimation. A *key innovation* of our work is the incorporation of view information into order-independent transmittance computation, thereby effectively leveraging the inherently view-dependent nature of 3DGS rendering. Moreover, the interaction between view direction and depth is effectively captured by the MLP, whereas such interactions are difficult to model using handcrafted depth-based functions. Table IV further demonstrates that incorporating view information (OIT+d+view) consistently outperforms the variant using only depth information (OIT+d).

Advantages over related works. (1) *Higher accuracy.* As shown in Table IV, our method consistently exhibits smaller degradation in PSNR and SSIM relative to weight-

TABLE IV
IMAGE QUALITY COMPARISON WITH BASELINE AND THE SOTA SOLUTION.

Scene	Bicycle	Bonsai	Counter	Garden	Kitchen	Room	Stump	Avg.
Base. PSNR↑	23.71	29.66	27.14	26.30	28.86	29.21	25.62	27.21
Weight-sum [18]	23.05	26.65	24.96	24.67	26.24	28.11	24.37	25.43
OIT+d	23.28	27.44	26.09	26.01	26.72	28.93	24.78	26.17
OIT+d+view	23.87	28.80	26.83	26.25	28.34	28.97	25.23	26.90
Base. SSIM↑	0.6684	0.9223	0.8782	0.8333	0.9022	0.8930	0.7200	0.8309
Weight-sum [18]	0.6604	0.8772	0.8240	0.6251	0.8692	0.8811	0.6809	0.7700
OIT+d	0.6643	0.8948	0.8668	0.8215	0.8745	0.8916	0.6826	0.8137
OIT+d+view	0.6810	0.9081	0.8747	0.8276	0.8986	0.8966	0.6975	0.8263
Base. LPIPS↓	0.3240	0.1623	0.2062	0.1232	0.1272	0.2171	0.2530	0.2017
Weight-sum [18]	0.2667	0.2037	0.2321	0.1930	0.1490	0.1964	0.2504	0.2122
OIT+d	0.2620	0.1785	0.1860	0.1037	0.1451	0.1859	0.2490	0.1869
OIT+d+view	0.2454	0.1581	0.1759	0.0969	0.1226	0.1844	0.2339	0.1739

sum approaches, while maintaining competitive LPIPS scores. (2) *Higher training efficiency and practicality.* Despite being MLP-based, our formulation remains lightweight, and it requires only approximately 30 minutes of additional training per scene within our training framework. In contrast, weight-sum approaches typically require training from scratch with reduced learning rates to ensure stability, resulting in longer convergence times. (3) *Lower hardware cost.* Importantly, our MLP-based OIT is not merely an algorithmic alternative; it is developed from a hardware–algorithm co-design perspective. Although it involves more MAC operations than weight-sum rendering, it reuses the MAC datapath and exponential units already required for rasterization; therefore, MLP inference incurs nearly negligible additional hardware overhead. For example, to match the throughput of our MLP-based OIT within the same accelerator, deploying weight-sum rendering would require additional division units, incurring an extra 0.363 mm^2 area and 341 mW power, whereas our reconfiguration introduces only 0.147 mm^2 area and 88 mW power overhead.

C. Hardware Ablation Study

Technique breakdown analysis. We evaluate the impact of each optimization through four design variants: (i) *Baseline (BS)*, consisting of a conventional 16×16 rasterization array and a 32-parallel bitonic sorting network [20] following [25]. (ii) *BS+AR*, where the baseline rasterization array is replaced by axis-shared rasterization (AR). (iii) *BS+AR+OIT*, which applies order-independent transmittance (OIT) to remove the sorting network. It conducts the MLP inference for decay factor computation, replacing the sorting. (iv) *BS+AR+OIT+IP*, which additionally integrates our interleaved pipeline (IP). All variants are compared under the same area budget for fairness. As shown in Fig. 13, axis-shared rasterization improves throughput by $1.37\times$ on average. Adding order-independent transmittance further boosts throughput to $2.16\times$, while the full optimization achieves a $2.27\times$ geometric mean throughput improvement.

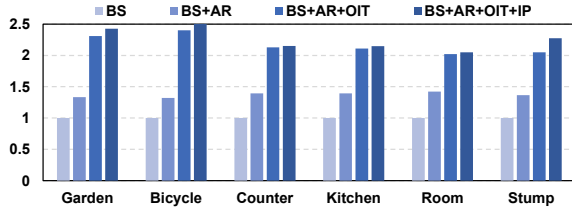


Fig. 13. Throughput of variants isolating each optimization.

Reconfiguration analysis and sorting comparison. We compare our reconfigurable design with a baseline PE array dedicated exclusively to rasterization, maintaining the same MAC count but without reconfigurability. Specifically, the baseline array comprises 6 MUL, 6 ADD, and 1 EXP unit per PE, whereas our reconfigurable array contains the same arithmetic units augmented by multiplexers to support reconfiguration. Experimental results in Fig. 14 (left) indicate that reconfiguration incurs only a 5% area overhead and a 6%

power overhead. The additional latency overhead is minimal, requiring two extra cycles—one for mode configuration and one for register clearing. Compared with a naive design using separate arrays for MLP inference and rasterization, our architecture delivers $1.91\times$ higher area efficiency (throughput/area) and $1.89\times$ higher energy efficiency (throughput/power), underscoring the effectiveness of the reconfigurable architecture. Relative to the 32-parallel bitonic sorting network [20] with hierarchical sorting [25], our reconfigurable PE array for MLP inference achieves a $21.1\times \sim 32.4\times$ speedup, as shown in Fig. 14 (right). This substantial speedup, combined with negligible quality degradation, demonstrates the effectiveness of order-independent transmittance as both an efficient and practical solution.

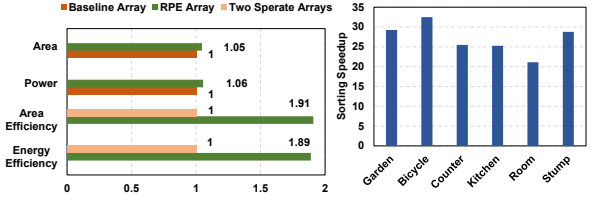


Fig. 14. Reconfiguration analysis and sorting speedup.

Tile schedule trajectory study. Fig. 15 (left) presents the average cache hit rate across scenes for three tile scheduling methods: the baseline trajectory, the Z-trajectory, and our generalized π -trajectory tile schedule. The baseline achieves a hit rate of 43%, the Z-trajectory 55%, and our method improves the hit rate to 62%. The corresponding off-chip access energy is shown on the right, with the configuration without cache normalized to 1. Thanks to the horizontal, vertical, and hierarchical locality utilized, our π -trajectory tile schedule achieves a $2.56\times$, $1.51\times$, and $1.23\times$ energy saving over no-cache setting, baseline trajectory, and Z-trajectory.

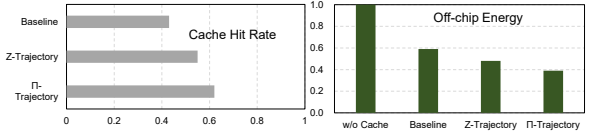


Fig. 15. Cache hit rate and energy comparison.

D. Comparison with Other Implementations

1) *Comparison with GPUs: Rasterization and sorting comparison.* As shown in Fig. 16 (left), our design achieves a rasterization speedup of $4.6 \sim 7.9\times$ over the edge GPU, with throughput exceeding 150 frames per second (FPS) as indicated on the secondary axis. This speedup stems from axis-shared rasterization, which avoids redundant computations and reduces the MAC count, while our dedicated hardware architecture sustains high parallelism and PE utilization. Fig. 16 (right) compares sorting latency on the edge GPU with that of our MLP-based OIT inference, which replaces the sorting process. It shows that even the naive pipeline achieves a speedup of $21 \sim 119\times$ over the edge GPU, as order-independent transmittance converts the originally expensive sorting process into a lightweight MAC operation that is

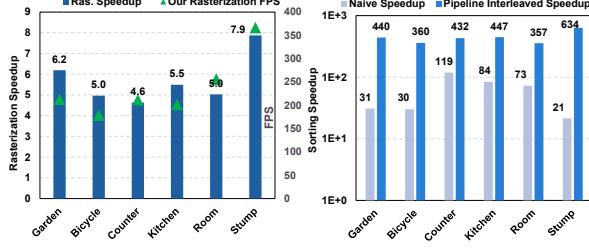


Fig. 16. Speedup of rasterization and sorting over edge GPU.

efficiently executed by our PE array. By resolving the memory-bound bottleneck, our fine-grained interleaved pipeline further enhances PE utilization, with a speedup over $300\times$.

Overall comparison: We combine our rasterization and sorting optimizations for comprehensive analysis. To simulate practical applications, we report end-to-end performance by executing the Gaussian projection step on the edge GPU, extracting latency and energy consumption, and integrating them into our evaluation. As shown in Table III, our design occupies a much smaller area and is implemented using a less advanced technology node, yet it still achieves substantial speedup and energy savings. Fig. 17 (top) shows that our combined optimizations achieve a $6.3 \sim 10.3\times$ speedup over the edge GPU and a $1.2 \sim 1.5\times$ speedup over the RTX 3090 desktop GPU. For end-to-end speedup, where Gaussian projection dominates latency, our design achieves a $4.0 \sim 5.5\times$ improvement over the edge GPU and a $1.1 \sim 1.4\times$ improvement over the desktop GPU. Fig. 17 (bottom) illustrates that our combined optimizations yield $16.2\times \sim 31.9\times$ energy savings over the edge GPU and $45.6 \sim 79.6\times$ savings over the desktop GPU. For end-to-end inference, the energy savings are $5.1 \sim 13.5\times$ over the edge GPU and $4.6 \sim 9.0\times$ over the desktop GPU. These improvements stem from our dedicated reconfigurable hardware design, MAC reduction via axis-shared rasterization, high parallelism with efficient PE utilization, and the adoption of the order-independent transmittance method.

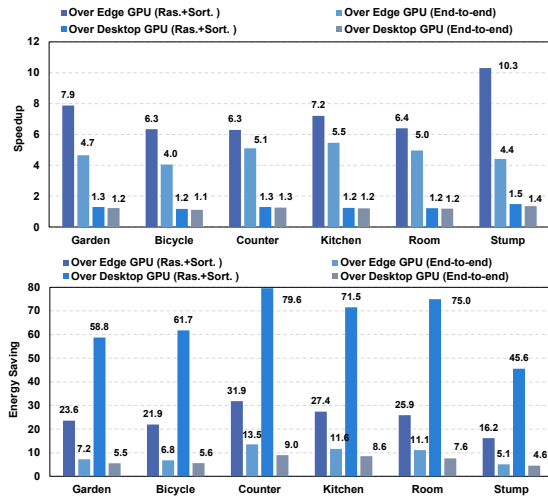


Fig. 17. Overall speedup and energy saving over GPUs.

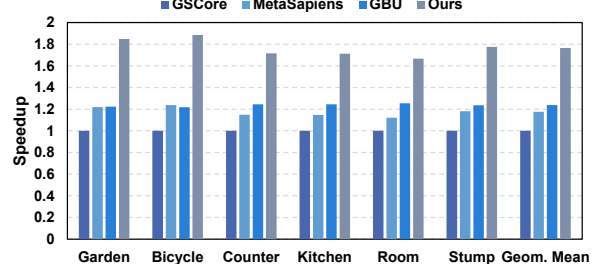


Fig. 18. Speedup over SOTA accelerators.

2) *Comparison with SOTA accelerators:* We compare our design with GScore [25], GBU [52], and MetaSapiens [29], with a focus on the sorting and rasterization stages. All designs are based on a 28 nm process and operate at 1 GHz. To ensure fairness, all designs are normalized to the same area budget¹. As shown in Fig. 18, taking GScore as the baseline, our design achieves a $1.67 \sim 1.88\times$ speedup, enabled by the dedicated axis-shared rasterization and order-independent transmittance. MetaSapiens [29] mitigates pipeline imbalance through tile merging and an incremental pipeline, but at the cost of additional buffers and complex control, while still leaving rasterization redundancy unaddressed. Our design achieves a $1.49 \sim 1.52\times$ speedup over MetaSapiens. Compared with GBU, our design achieves a $1.33 \sim 1.55\times$ speedup. Although GBU also reduces rasterization MACs through sequential computation, it lacks sorting optimization. Furthermore, sequential computation introduces pixel dependencies that limit parallelism scalability, leading to additional overhead.

E. GPU-implemented Optimization Evaluation

1) *GPU-implemented axis-shared rasterization:* **Implementation.** To further clarify the benefits of our co-designed accelerator, we implement axis-shared rasterization on an NVIDIA RTX 3090 GPU. Axis-shared rasterization consists of three stages: shared-term computation, broadcast, and combination, as described in Sec. III. To map this structure onto the GPU, we assign one 16×16 thread block to each tile. Within each block, we introduce an explicit shared-term computation stage, where threads collaboratively compute the X-axis and Y-axis shared terms. These intermediate results are stored in shared memory. After synchronization, all threads reuse the shared terms to perform the combination stage in parallel.

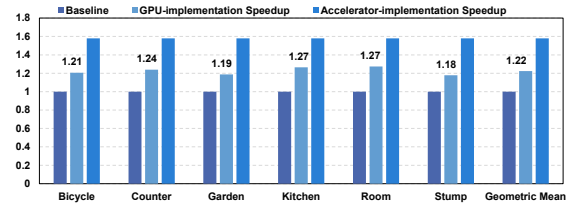


Fig. 19. The effect of GPU-implemented axis-shared optimization.

¹Since GScore and MetaSapiens propose techniques to reduce the Gaussian count, and these techniques are compatible with our design. To guarantee fairness, we assume that all designs employ the same Gaussian reduction technique as GScore.

Fig. 19 shows the performance improvement of a GPU-based implementation of axis-shared rasterization across scenes. The baseline is the original Gsplat implementation [51]. For reference, we include the performance of our accelerator implementation (right), which serves as an architectural upper bound. Because axis-shared rasterization reduces MAC count and hardware area, we normalize speedup by comparing latency under an equivalent area budget. The GPU implementation achieves a geometric mean speedup of 22%, substantially lower than the nearly 60% speedup delivered by our dedicated accelerator. Because rasterization is a non-GEMM workload, it cannot effectively utilize Tensor Cores and is therefore executed on CUDA cores. Compared with our accelerator, the GPU mapping exhibits limitations in both computational structure and memory behavior.

Computational limitations. (i) Axis-shared rasterization consists of an $O(L)$ shared-term stage followed by an $O(L^2)$ combination stage. This imbalance leads to substantial thread underutilization during the $O(L)$ stage. In contrast, our accelerator exploits the $O(L)$ edge and $O(L^2)$ area relationship of the PE array, mapping shared-term computation to PE lines and the combination stage to the full array. This spatial decomposition enables near-full utilization across both stages. (ii) The GPU MAC configuration cannot fully accommodate the requirements of rasterization. The shared-term stage requires more multiplications than additions, whereas the combination stage involves addition-before-multiplication patterns that deviate from the standard fused multiply-add (FMA) pipeline. GPUs employ balanced multiplier-adder ratios and fixed FMA pipelines, making such irregular arithmetic sequences inefficient. Moreover, frequent exponential operations further increase computational pressure. In contrast, our accelerator adopts a dedicated datapath, allowing the reduced MAC count to translate directly into area and latency savings.

Memory limitations. (i) The shared terms must be written to and read from shared memory between stages, incurring non-negligible synchronization and access overhead. In contrast, our accelerator fully fuses the three stages into a register-to-register datapath without intermediate buffer or memory accesses. (ii) The GPU-based axis-shared rasterization trades additional storage for reduced MAC count. Because existing 3DGS GPU kernels already heavily utilize register files and shared memory, the additional reuse of shared terms increases register and shared-memory pressure, raising the risk of register spilling. In contrast, our dedicated accelerator pipeline carefully allocates registers, enabling shared terms to be produced and consumed in place, without extra storage. Overall, these computational and memory constraints highlight the limitations of deploying axis-shared rasterization on GPUs, further motivating a dedicated accelerator design.

2) *GPU-implemented MLP-based OIT*: The MLP-based OIT is also implemented on an NVIDIA RTX 3090 GPU using cuBLAS [37], and its latency is compared against the Radix sorting [31] implementation in Gsplat [51] as the baseline. As shown in Fig. 20, the GPU-implemented MLP-based OIT is slower, and exhibits a geometric mean latency of $1.59\times$

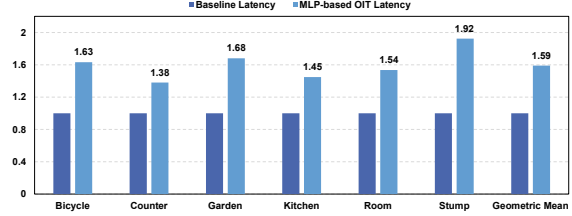


Fig. 20. Latency comparison between baseline sorting and MLP-based OIT.

that of the baseline sorting approach. Although our MLP is extremely lightweight, its inference on the GPU becomes memory bound. As discussed in Sec. V-B, the small MLP exhibits low arithmetic intensity, thereby limiting throughput under the GPU execution model. As a result, the theoretical arithmetic simplicity of the MLP does not translate into practical latency reduction on the GPU.

In our accelerator, the MLP-based OIT solves the challenge of sorting on edge devices, as described in Sec. II-B2. It is mapped onto the same unified PE array through a re-configurable design, incurring negligible additional hardware overhead while eliminating inter-stage pipeline imbalance. Furthermore, the fine-grained interleaved pipeline effectively hides memory latency and mitigates the memory-bound bottleneck. These architectural optimizations collectively enable substantial speedup over the baseline GPU sorting approach, as reported in Sec. VI-D, further demonstrating the motivation of our dedicated accelerator design.

F. Applicability to Dynamic Scenes

Although our design primarily targets static 3DGS rendering, it is also meaningful to study the applicability of order-independent transmittance (OIT) to dynamic scenarios. We extend OIT from 1D image composition to 3D Gaussian splatting by incorporating view information, a natural question is whether it remains effective for dynamic scenes. To investigate this, we evaluate our MLP-based OIT on the Neu3D dataset [27], a widely used benchmark for real-world dynamic view synthesis, featuring high-resolution sequences (2704×2028) and spanning 10 seconds (300 frames). We adopt 4DGS [50] as the baseline for dynamic scene modeling. 4DGS models dynamic scenes using 4D primitives that generate standard 3D Gaussians at each timestamp, making the per-frame rendering pipeline identical to that of static 3DGS. To account for temporal variations, we update the weights of the 10-parameter MLP every 30 frames. For a typical 300-frame sequence, this produces 10 independent sets of MLP parameters covering the entire sequence.

TABLE V
EVALUATION OF OUR OIT APPLIED TO DYNAMIC SCENES.

	Cook Spinach		Cut Beef		Flame Steak		Average	
	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM	PSNR	SSIM
Baseline	32.88	0.9572	32.75	0.9575	32.78	0.9552	32.80	0.9566
Our OIT	32.46	0.9566	32.29	0.9571	32.31	0.9550	32.35	0.9562

As shown in Table V, our MLP-based OIT maintains high fidelity, with only a 0.45 PSNR drop compared with the baseline. This result indicates that each localized MLP can

effectively capture evolving occlusion relationships within its time window. The additional overhead is minimal, requiring only 10×10 extra parameters, making the approach well suited for edge deployment. Looking forward, dynamic scene rendering is expected to involve longer sequences and richer temporal variations. While our results demonstrate the feasibility of the proposed method for dynamic scenes, extending it to longer sequences or scenes with more drastic geometric changes may require further adaptation, which we leave for future work.

VII. RELATED WORK

Efficient 3D Gaussian Splatting algorithm. Recent studies aim to accelerate rendering or reduce memory overhead in 3DGS primarily by pruning Gaussian counts [7]–[9], [12], [36], [53], while acceleration of the sorting process remains relatively underexplored. A recent work [18] proposes monotonically decreasing depth-based functions to enable nearly sort-free rendering, demonstrating that learnable functions can replace traditional sorting. However, this method relies solely on depth input and manually designed function forms, limiting expressiveness and generalization across diverse scenes. A detailed discussion was provided in Sec. VI-B.

3D Gaussian Splatting accelerators. To meet the real-time rendering demands of edge devices, several specialized hardware accelerators have recently been proposed [21], [25], [29], [39], [46], [52]. GBU [52] introduces a rasterization module for edge GPUs designed to reduce MAC operations. However, our approach fundamentally differs in both motivation and implementation. GBU reduces MACs through spatial transformation and eigenvalue decomposition, whereas our method identifies computational redundancy and introduces axis-shared computation to eliminate it, resulting in a distinct hardware architecture. In terms of parallelism, GBU performs differential computations row-wise by reusing consecutive pixels, which introduces dependencies that limit intra-tile parallelism. In contrast, our method enables fully parallel pixel processing within each tile, substantially improving scalability and throughput. GSCore [25] streamlines the Gaussian splatting pipeline by reducing the number of Gaussians assigned to tiles using a shape-aware intersection test and introduces hierarchical sorting through a sorting network, thereby accelerating both rasterization and sorting. MetaSapiens [29] leverages efficiency-aware pruning and the low visual acuity of the human periphery to relax rendering quality, improve rendering speed, and accelerate foveated rendering. These Gaussian count reduction techniques [25], [29] are compatible with our design, but our approach diverges in two key aspects. First, we identify inherent redundancy in rasterization and propose axis-shared rasterization, supported by a highly parallel architecture that reduces the MAC count without sacrificing generality. Second, we optimize the sorting process through algorithm–hardware co-design: sorting is replaced by order-independent transmittance at the algorithmic aspect, while our reconfigurable array with an interleaved pipeline reduces MLP inference latency to a negligible level. In addition, several accelerators focus on domain-specific 3DGS applications. For

example, GauSPU [48] introduces a co-processor for 3DGS-based SLAM, while GsArch [16] alleviates memory bottlenecks to improve training efficiency. Lumina [11] targets moving-view scenarios and reduces sorting and rasterization costs by reusing results across consecutive frames.

MAC reduction in traditional splatting. FastSplat [19] reduces MAC operations via differential computation, reusing intermediate results between consecutive pixels and updating only incremental terms. While effective for sequential software implementations, this formulation introduces inter-pixel dependencies that constrain spatial parallelism and complicate scalable hardware mapping. In contrast, our axis-shared rasterization removes redundancy by precomputing and sharing common terms along the X- and Y-axes within each tile, without introducing sequential dependencies. This design enables full spatial parallelism across processing elements while reducing redundant arithmetic operations. Moreover, FastSplat targets traditional splatting, whereas our work focuses on anisotropic Gaussian rasterization in modern 3DGS pipelines with different computational structures.

GPU-based 3DGS acceleration. Orthogonal to dedicated 3DGS accelerator design, many studies focus on GPU-based acceleration by employing efficient scheduling strategies and CUDA kernel optimization [4], [10], [14], [15], [28], [44], [54]. For instance, FlashGS [10] and Speedy-Splat [15] develop more accurate Gaussian-intersection detection primitives, while Balanced 3DGS [14] reduces thread idling and mitigates load-imbalance issues through dynamic workload distribution. Additionally, TC-GS [28] transforms the α -computation into matrix multiplication and utilizes otherwise idle Tensor Cores to accelerate the rendering pipeline. GauRast [26] augments existing GPU triangle rasterizers to support Gaussian-based rendering. By leveraging the structural similarities between traditional graphics pipelines and 3DGS, it achieves substantial energy savings and throughput gains for edge deployment.

VIII. CONCLUSION

This work presents an architecture–algorithm co-design to enable real-time 3D Gaussian Splatting on resource-constrained platforms. We identify two overlooked challenges, computational redundancy in rasterization and scalability and imbalance issues in sorting, and address them through novel solutions. First, the proposed *axis-shared rasterization* eliminates redundant computations by sharing intermediate terms across processing elements, reducing MAC operations by 38%. Second, our *order-independent transmittance* method bypasses explicit sorting with a lightweight MLP, mitigating pipeline bottlenecks while maintaining rendering quality. Finally, we develop a unified and reconfigurable hardware accelerator that sustains high utilization across rasterization and MLP inference. This work provides a practical and architecture-friendly foundation for efficient 3DGS acceleration, enabling broader adoption of real-time and efficient rendering on resource-constrained systems.

REFERENCES

- [1] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-nerf 360: Unbounded anti-aliased neural radiance fields," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 5470–5479.
- [2] L. Carpenter, "The a-buffer, an antialiased hidden surface method," in *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*, 1984, pp. 103–108.
- [3] K. Chandrasekar, C. Weis, Y. Li, B. Akesson, N. Wehn, and K. Goossens, "Drampower: Open-source dram power & energy estimation tool," URL: <http://www.drampower.info>, vol. 22, 2012.
- [4] S. Durvasula, A. Zhao, F. Chen, R. Liang, P. K. Sanjaya, Y. Guan, C. Giannoula, and N. Vijaykumar, "Arc: Warp-level adaptive atomic reduction in gpus to accelerate differentiable rendering," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 64–83. [Online]. Available: <https://doi.org/10.1145/3669940.3707238>
- [5] E. Enderton, E. Sintorn, P. Shirley, and D. Luebke, "Stochastic transparency," in *Proceedings of the 2010 ACM SIGGRAPH symposium on Interactive 3D Graphics and Games*, 2010, pp. 157–164.
- [6] C. Everitt, "Interactive order-independent transparency," *White paper, nVIDIA*, vol. 2, no. 6, p. 7, 2001.
- [7] Z. Fan, K. Wang, K. Wen, Z. Zhu, D. Xu, Z. Wang *et al.*, "Lightgaussian: Unbounded 3d gaussian compression with 15x reduction and 200+ fps," *Advances in neural information processing systems*, vol. 37, pp. 140 138–140 158, 2024.
- [8] G. Fang and B. Wang, "Mini-splatting: Representing scenes with a constrained number of gaussians," in *European Conference on Computer Vision*. Springer, 2024, pp. 165–181.
- [9] —, "Mini-splatting2: Building 360 scenes within minutes via aggressive gaussian densification," *arXiv preprint arXiv:2411.12788*, 2024.
- [10] G. Feng, S. Chen, R. Fu, Z. Liao, Y. Wang, T. Liu, Z. Pei, H. Li, X. Zhang, and B. Dai, "Flashgs: Efficient 3d gaussian splatting for large-scale and high-resolution rendering," 2024. [Online]. Available: <https://arxiv.org/abs/2408.07967>
- [11] Y. Feng, W. Lin, Y. Cheng, Z. Liu, J. Leng, M. Guo, C. Chen, S. Sun, and Y. Zhu, "Lumina: Real-time neural rendering by exploiting computational redundancy," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 1925–1939. [Online]. Available: <https://doi.org/10.1145/3695053.3731003>
- [12] S. Girish, K. Gupta, and A. Shrivastava, "Eagles: Efficient accelerated 3d gaussians with lightweight encodings," in *European Conference on Computer Vision*. Springer, 2024, pp. 54–71.
- [13] F. Gray, "Pulse code communication," *United States Patent Number 2632058*, 1953.
- [14] H. Gui, L. Hu, R. Chen, M. Huang, Y. Yin, J. Yang, Y. Wu, C. Liu, Z. Sun, X. Zhang, and K. Zhan, "Balanced 3dgs: Gaussian-wise parallelism rendering with fine-grained tiling," 2025. [Online]. Available: <https://arxiv.org/abs/2412.17378>
- [15] A. Hanson, A. Tu, G. Lin, V. Singla, M. Zwicker, and T. Goldstein, "Speedy-splat: Fast 3d gaussian splatting with sparse pixels and sparse primitives," 2025. [Online]. Available: <https://arxiv.org/abs/2412.00578>
- [16] H. He, G. Li, F. Liu, L. Jiang, X. Liang, and Z. Song, "GSArch: Breaking Memory Barriers in 3D Gaussian Splatting Training via Architectural Support," in *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025.
- [17] D. Hilbert and D. Hilbert, "Über die stetige abbildung einer linie auf ein flächenstück," *Dritter Band: Analysis: Grundlagen der Mathematik-Physik Verschiedenes: Nebst Einer Lebensgeschichte*, pp. 1–2, 1935.
- [18] Q. Hou, R. Rauwendael, Z. Li, H. Le, F. Farhadzadeh, F. Porikli, A. Bourd, and A. Said, "Sort-free gaussian splatting via weighted sum rendering," *arXiv preprint arXiv:2410.18931*, 2024.
- [19] J. Huang, K. Mueller, N. Shareef, and R. Crawfis, "Fastsplats: Optimized splatting on rectilinear grids," in *Proceedings Visualization 2000. VIS 2000 (Cat. No. 00CH37145)*. IEEE, 2000, pp. 219–226.
- [20] M. F. Ionescu and K. E. Schauser, "Optimizing parallel bitonic sort," in *Proceedings 11th International Parallel Processing Symposium*. IEEE, 1997, pp. 303–309.
- [21] H. Joo, S. Kim, J. Park, J. Ryu, and H.-J. Yoo, "A 51.2 fps real-time 3dgs-slam accelerator using diagonal feeding with symmetric alpha reuse and voxel-based 3d gaussian cache management," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, 2025, pp. 1–5.
- [22] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Trans. Graph.*, vol. 42, no. 4, pp. 139–1, 2023.
- [23] M. Khan, H. Fazlali, D. Sharma, T. Cao, D. Bai, Y. Ren, and B. Liu, "Autosplat: Constrained gaussian splatting for autonomous driving scene reconstruction," *arXiv preprint arXiv:2407.02598*, 2024.
- [24] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A fast and extensible dram simulator," *IEEE Computer architecture letters*, vol. 15, no. 1, pp. 45–49, 2015.
- [25] J. Lee, S. Lee, J. Lee, J. Park, and J. Sim, "Gscore: Efficient radiance field rendering via architectural support for 3d gaussian splatting," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 497–511.
- [26] S. Li, B. Keller, Y. C. Lin, and B. Khailany, "Gaurast: Enhancing gpu triangle rasterizers to accelerate 3d gaussian splatting," in *2025 62nd ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2025, pp. 1–7.
- [27] T. Li, M. Slavcheva, M. Zollhoefer, S. Green, C. Lassner, C. Kim, T. Schmidt, S. Lovegrove, M. Goesele, R. Newcombe *et al.*, "Neural 3d video synthesis from multi-view video," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022, pp. 5521–5531.
- [28] Z. Liao, J. Ding, S. Cui, R. Gong, B. Hu, Y. Wang, H. Li, X. Zhang, H. Wang, and R. Fu, "Tc-gs: A faster gaussian splatting module utilizing tensor cores," 2025. [Online]. Available: <https://arxiv.org/abs/2505.24796>
- [29] W. Lin, Y. Feng, and Y. Zhu, "Metasapiens: Real-time neural rendering with efficiency-aware pruning and accelerated foveated rendering," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, ser. ASPLOS '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 669–682. [Online]. Available: <https://doi.org/10.1145/3669940.3707227>
- [30] M. McGuire and L. Bavoil, "Weighted blended order-independent transparency," *Journal of Computer Graphics Techniques*, vol. 2, no. 4, 2013.
- [31] D. G. Merrill and A. S. Grimshaw, "Revisiting sorting for gpgpu stream architectures," in *Proceedings of the 19th international conference on Parallel architectures and compilation techniques*, 2010, pp. 545–546.
- [32] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "Nerf: Representing scenes as neural radiance fields for view synthesis," *Communications of the ACM*, vol. 65, no. 1, pp. 99–106, 2021.
- [33] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," 1966.
- [34] C. Münstermann, S. Krumpfen, R. Klein, and C. Peters, "Moment-based order-independent transparency," *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, vol. 1, no. 1, pp. 1–20, 2018.
- [35] N. Muralimanohar, R. Balasubramanian, and N. P. Jouppi, "Cacti 6.0: A tool to model large caches," *HP laboratories*, vol. 27, p. 28, 2009.
- [36] M. Niemeyer, F. Manhardt, M.-J. Rakotosaona, M. Oechsle, D. Duckworth, R. Gosula, K. Tateno, J. Bates, D. Kaeser, and F. Tombari, "Radsplat: Radiance field-informed gaussian splatting for robust real-time rendering with 900+ fps," *arXiv preprint arXiv:2403.13806*, 2024.
- [37] NVIDIA, "NVIDIA cuBLAS," 2026, a GPU-accelerated library for accelerating AI and HPC applications, providing implementations of BLAS routines and GEMM APIs optimized for NVIDIA GPUs. [Online]. Available: <https://developer.nvidia.com/cublas>
- [38] NVIDIA Corporation, "Getting started with nvidia jetson orin nano developer kit," <https://developer.nvidia.com/embedded/learn/get-started-jetson-orin-nano-devkit>, 2024, accessed: 2026-05.
- [39] M. Pei, G. Li, J. Si, Z. Zhu, Z. Mo, P. Wang, Z. Song, X. Liang, and J. Cheng, "Gcc: A 3dgs inference architecture with gaussian-wise and cross-stage conditional processing," 2025. [Online]. Available: <https://arxiv.org/abs/2507.15300>
- [40] T. Porter and T. Duff, "Compositing digital images," in *Proceedings of the 11th annual conference on Computer graphics and interactive techniques*, 1984, pp. 253–259.

- [41] D. P. Rodgers, "Improvements in multiprocessor system design," *ACM SIGARCH Computer Architecture News*, vol. 13, no. 3, pp. 225–231, 1985.
- [42] A. R. Smith, "Alpha and the history of digital compositing," Citeseer, Tech. Rep., 1995.
- [43] Synopsys, Inc., "DesignWare Library," <https://www.synopsys.com/designware-ip/soc-infrastructure-ip/designware-library.html>.
- [44] M. Tao, Y. Zhou, H. Xu, Z. He, Z. Yang, Y. Zhang, Z. Su, L. Xu, Z. Ma, R. Fu, H. Li, X. Zhang, and J. Zhai, "Gs-cache: A gs-cache inference framework for large-scale gaussian splatting models," 2025. [Online]. Available: <https://arxiv.org/abs/2502.14938>
- [45] X. Tu, B. Kerbl, and F. de la Torre, "Fast and robust 3D Gaussian splatting for virtual reality," in *SIGGRAPH Asia 2024 Posters*, 2024, pp. 1–3.
- [46] Y. Wang, Y. Li, J. Chen, J. Yu, and K. Wang, "Famers: An fpga accelerator for memory-efficient edge-rendered 3d gaussian splatting," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.
- [47] S. Williams, A. Waterman, and D. Patterson, "Roofline: an insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009.
- [48] L. Wu, H. Zhu, S. He, J. Zheng, C. Chen, and X. Zeng, "Gauspu: 3d gaussian splatting processor for real-time slam systems," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2024, pp. 1562–1573.
- [49] J. Xu, Z. Li, B. Du, M. Zhang, and J. Liu, "Reluplex made more practical: Leaky relu," in *2020 IEEE Symposium on Computers and communications (ISCC)*. IEEE, 2020, pp. 1–7.
- [50] Z. Yang, H. Yang, Z. Pan, and L. Zhang, "Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting," *arXiv preprint arXiv:2310.10642*, 2023.
- [51] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik *et al.*, "gsplat: An open-source library for gaussian splatting," *Journal of Machine Learning Research*, vol. 26, no. 34, pp. 1–17, 2025.
- [52] Z. Ye, Y. Fu, J. Zhang, L. Li, Y. Zhang, S. Li, C. Wan, C. Wan, C. Li, S. Prathipati, and Y. C. Lin, "Gaussian blending unit: An edge gpu plugin for real-time gaussian-based rendering in ar/vr," in *Proceedings of the 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 2025.
- [53] Z. Ye, C. Wan, C. Li, J. Hong, S. Li, L. Li, Y. Zhang, and Y. C. Lin, "3d gaussian rendering can be sparser: Efficient rendering via learned fragment pruning," *Advances in Neural Information Processing Systems*, vol. 37, pp. 5850–5869, 2024.
- [54] Y. Yuan and Q. He, "Efficient differentiable hardware rasterization for 3d gaussian splatting," 2025. [Online]. Available: <https://arxiv.org/abs/2505.18764>
- [55] H. Zhai, X. Zhang, B. Zhao, H. Li, Y. He, Z. Cui, H. Bao, and G. Zhang, "Splatloc: 3D Gaussian splatting-based visual localization for augmented reality," *IEEE Transactions on Visualization and Computer Graphics*, 2025.
- [56] S. Zhu, G. Wang, X. Kong, D. Kong, and H. Wang, "3D Gaussian splatting in robotics: A survey," *arXiv preprint arXiv:2410.12262*, 2024.